Service contract — v1.0 (RE-00d, DS-017)

> **Status: proposed, for verification and Aravinth's confirmation.** DS-003 RE-00d: "*Agree the request and response shape between the app and the recommendation service, and agree the single endpoint all twelve events are written to.*" Nothing on the capture register can be marked done until that endpoint exists.

The twelve events and their fields are the **DS-003 capture register**: events E1 to E12, fields F01 to F59, with the register's own names. Where this contract adds something the register does not have, it is listed under *Beyond the register* along with the reason.

The contract is also enforced in code, in `reco/domain/service_contract.py`. One test reads the capture register straight out of the workbook and fails if an event gains or loses a field. SOCO_BE validates against `docs/service_contract.schema.json`, which is generated from the same models.

The two calls

Call	Direction	What it is
`POST /v1/recommendations`	SOCO_BE → service	The request and response shape: a surface asks for a list
`POST /v1/events`	SOCO_BE → service	**The one endpoint all twelve events are written to**

The Flutter app calls SOCO_BE, as it does today. SOCO_BE calls the service. The ingest endpoint takes a service token, and an app binary cannot keep a token secret.

`POST /v1/recommendations`

Request

```
{
  "contract_version": "1.0",
  "user_id": "usr_4471",
  "surface": "feed",
  "limit": 20,
  "page": 1,
  "is_business_account": false,
  "refresh_seed": "a91f"
}
```

These are the fields `/dashboard-feed` already receives, spelled the way the register spells them. `surface` is F02 and must be one of `feed`, `product` or `opportunity`. `session_id` (F04) is not included: the register

defers it, and the app has no session identifier.

Response — 200

```
{
  "contract_version": "1.0",
  "request_id": "req_8f2a1c",
  "policy_id": "wave1a_v1",
  "server_ts": "2026-11-04T09:15:22.140Z",
  "is_exploration": false,
  "personalised": true,
  "items": [
    { "item_id": "pst_3390", "item_type": "post", "position": 1 },
    { "item_id": "prd_5512", "item_type": "product", "position": 2 }
  ]
}
```

Field	Register	Rule
`request_id`	F01, R1	The client keeps it and echoes it on every event from this list
`policy_id`	F06, R3	The configuration version and, on the three two-method slots, the method that served (PT-10)
`server_ts`	F07, R7	UTC
`is_exploration`	F14, R5	Set by the service for the randomised slice. The client echoes it and never computes it
`personalised`	—	`false` when the list is not built from this user's own profile. Do not label such a list "Recommended for you"
`items[].position`	F11	Starts at 1, with no gaps

`candidate_ids` (F05) is **not** returned to the client. The service writes it on the `recommendation_request` event.

Timeout: 200 ms (PT-07). On a timeout or error, SOCO_BE serves the delivered ordering. It then generates its own `request_id`, stamps `policy_id: "delivered"`, and sends a `recommendation_request` event itself. Impressions from a fallback list therefore still join back to a request, and that request says the engine did not order the list.

POST /v1/events

Request

```
POST /v1/events
Authorization: Bearer <INGEST_TOKEN>
Content-Type: application/json
```

```
{
  "contract_version": "1.0",
  "sent_at": "2026-11-04T09:15:40.002Z",
  "events": [
    {
      "event_id": "0b6f6a52-3c2e-4a8e-9a57-2f1c7f0f4d10",
      "event_name": "recommendation_impression",
      "request_id": "req_8f2a1c",
      "item_id": "pst_3390",
      "item_type": "post",
      "position": 1,
      "viewport_entered": true,
      "dwell_ms": 2400,
      "is_exploration": false,
      "client_ts": "2026-11-04T14:45:22+05:30"
    },
    {
      "event_id": "5d0e2f7e-8a41-4f5b-b1f2-6f3f2b9c1e77",
      "event_name": "order_state_changed",
      "order_id": "ord_2210",
      "new_state": "Cancelled",
      "reason_code": null,
      "state_ts": "2026-11-09T11:02:00Z"
    }
  ]
}
```

A request holds 1 to 500 events. Every event carries `event_id` and `event_name`, followed by its register fields.

The twelve events

ID	`event_name`	Fields	Register decision
E1	`recommendation_request`	`request_id` · `surface` · `user_id` · `session_id` · `candidate_ids` · `policy_id` · `server_ts`	Accepted
E2	`recommendation_impression`	`request_id` · `item_id` · `item_type` · `position` · `viewport_entered` · `dwell_ms` · `is_exploration` · `client_ts`	Accepted
E3	`recommendation_click`	`request_id` · `item_id` · `position` · `server_ts`	Accepted
E4	`add_to_cart`	`request_id` · `product_id` · `quantity` · `user_id`	Accepted
E5	`order_placed`	`order_id` · `request_id` · `line_items` · `promoter_id` · `order_value` · `server_ts` · `checkout_id`	Accepted
E6	`order_state_changed`	`order_id` · `new_state` · `reason_code` · `state_ts`	Accepted
E7	`payment_state_changed`	`order_id` · `new_state` · `state_ts`	Needs discussion
E8	`commission_state_changed`	`commission_id` · `promoter_id` · `order_id` · `new_state` · `amount` · `state_ts`	Accepted
E9	`social_interaction`	`user_id` · `target_id` · `action` · `request_id` · `server_ts`	Accepted
E10	`opportunity_state_changed`	`opportunity_id` · `promoter_id` · `campaign_id` · `new_state` · `state_ts`	Deferred
E11	`search_performed`	`user_id` · `query_text` · `result_count` · `server_ts`	Accepted
E12	`identity_stitched`	`anonymous_id` · `user_id` · `stitch_ts`	Needs discussion

The contract defines a shape for all twelve, whatever each event's decision. A deferred event then arrives later as implementation work, not as a new negotiation.

Allowed values

Field	Values
`surface`	`feed` · `product` · `opportunity`
`item_type`	`post` · `product` · `promotion` · `promoter` · `brand`
`order_state_changed.new_state`	`Accepted` · `Confirmed` · `Shipped` · `Delivered` · `Cancelled` · `Rejected`, the `order_status` values (Q06)
`payment_state_changed.new_state`	`settled` · `failed` · `charged_back`
`commission_state_changed.new_state`	`attributed` · `confirmed` · `reversed`
`social_interaction.action`	`like` · `save` · `share` · `repost` · `follow`
`opportunity_state_changed.new_state`	`accepted` · `completed` · `withdrawn` · `expired`

Rules

- **Every field listed is required.** A field that can be empty is sent as `null`, never left out. A missing key means the caller has a bug; `null` means "not known".
- **These fields can be `null`:** `session_id` (deferred) · `dwell_ms` (not measured) · `request_id` on `add_to_cart`, `order_placed` and `social_interaction` (not reached from a list) · `promoter_id` and `checkout_id` (direct order) · `reason_code` · `campaign_id` (until Q02).
- **Server times are UTC** (R7): `server_ts`, `state_ts`, `stitch_ts`. A time with any other offset is rejected.
- `client_ts` **must carry the device's offset** (R7), for example `+05:30`. It is stored exactly as sent and never converted.
- **Any field not listed is rejected**, and that includes a `weight`. Weights come from P-06 only.

Response — `202 Accepted`

This is the response you would get if the impression above had sent `"position": 0`:


```
{
  "contract_version": "1.0",
  "received_at": "2026-11-04T09:15:40.118Z",
  "accepted": 1,
  "duplicates": 0,
  "rejected": [
    { "index": 0, "event_id": "0b6f6a52-3c2e-4a8e-9a57-2f1c7f0f4d10",
      "errors": ["recommendation_impression.position: Input should be greater than or equal to
1"] }
  ]
}
```

- `accepted + duplicates + rejected` always equals the number of events sent.
- **Each event is judged on its own.** A bad impression does not throw away a good order sent in the same batch.
- **Accepted means stored, not scored.** The server adds `received_at`, which is kept separate from the event's own time, as R7 requires.

Status	When	What SOCO_BE does
`400`	The request wrapper is unusable: bad JSON, unknown `contract_version`, 0 or more than 500 events	Nothing was stored. Fix the caller
`401`	Token missing or wrong	Nothing was stored
`503`	The event store cannot be reached	Retry later with the **same** `event_id`s. Duplicates are counted, not stored twice

Never block a user request on this call: send it after responding, or from a queue. Use a 2-second timeout; a retry is always safe.

Accepted events go into `ml_events`. It has a unique index on `event_id` and indexes for a user's history and for joining on `request_id`.

Beyond the register

Addition	On	Why
`event_id` (UUID)	every event	The idempotency key. Without it, a timeout followed by a retry counts every event in the batch twice
`event_name`	every event	Says which of the twelve an event is. The register's "Event name" column, carried as a field
`user_id`	`add_to_cart`	The register links a cart add to a user only through `request_id`, which is null whenever the product was not reached from a list. The engagement call already sends `user_id`

What M1 reads from these events

Event	M1 signal (P-06)	Condition
`recommendation_impression`	view	`viewport_entered` is true and `dwell_ms` ≥ P-02 (3 s). A `null` dwell scores nothing and is counted separately
`recommendation_click`	click	—
`add_to_cart`	cart	—
`order_placed`	purchase	Credited at order creation, not at Delivered
`order_state_changed`	purchase, **negated**	Only when `new_state` is `Cancelled` or `Rejected`. Aged from the original `order_placed.server_ts`, never from `state_ts`
`search_performed`	search	Not computable as registered: see point 5 below

The rest are captured for the other model families. M1 does not read them.

Points that need a ruling

1. Position counts from 1. The register says positions start at 1 (F11, F18). Aravinth's F11 note describes a 0-based index. The contract uses 1, and rejects 0.

2. Should impressions be sent for items never seen? E2's check says one event per item that entered the screen, and *none* for items never reached. F12's check says `viewport_entered` must show *both* true and false. Both cannot hold. The contract accepts both until this is ruled on.

3. `request_id` can be `null` on three events. Register checks F20, F24 and F45 say "no nulls". That is right for events that came from a served list. It cannot hold for a cart add from search, a direct order, or a follow from a profile page. The contract allows `null` on those three and nowhere else.

4. Surface values. Aravinth's F02 note says `feedType` already carries the surface. Its current values need mapping onto `feed`, `product` and `opportunity`, or those values need adding here. The register allows no free text.

5. Search cannot feed M1 as registered. P-06 credits a search to the categories of its *result set*. E11 carries `query_text` and `result_count` but no result categories, so M1-15 stays not computable. The proposal is to add `result_categories` to E11.

6. Item types. F10 lists product, promotion, promoter or brand. F32 lists product, post, promoter or brand. Feed items are posts, so the contract accepts both lists combined.

7. Returns (D1). A return is recorded as `return_requested` and `return_requested_at` on the order, with `order_status` unchanged, so it is not an `order_state_changed`. The service reads those two fields from the order until an event is agreed for them.

Not built yet

This is the contract, not the routes. Both routes are built once it is agreed. Two things are needed first:

- **`ml_events` is added to the `mlWriter` role**, next to `ml_user_interest_scores` in HANDOFF.md §3.
- **An `INGEST_TOKEN` is agreed** and stored on both sides.